

Agente Aspiradora

Simulación de un agente
reactivo



PRESENTADO POR

Johann Ricaurte
Cristian Martinez

Representación del Ambiente

MATRIZ 4X4

Cada casilla se representa como una matriz de **4x4**, que contiene **16 píxeles**. Esto permite analizar y gestionar la limpieza en un espacio definido y limitado.

VALORES DE PÍXELES

Cada píxel tiene un valor que varía entre **0 y 255**, donde los valores bajos indican suciedad y los altos representan limpieza. Esta escala define el estado de cada casilla.

Evaluación de Suciedad

DEFINICIÓN DE LIMPIEZA

Un pixel es considerado limpio si su valor es **mayor o igual a 200**, lo que indica que presenta un estado adecuado. Este criterio es fundamental para la evaluación.

CÁLCULO DE SUCIEDAD

La casilla es clasificada como sucia si el **porcentaje de píxeles sucios** supera el 50 por ciento. Este cálculo es esencial para determinar las acciones del agente aspiradora.

Reglas del Agente

CONDICIÓN DE SUCIEDAD

Si la casilla está sucia, el agente realiza la limpieza, eliminando la suciedad acumulada y mejorando el estado del ambiente en que opera.

MOVIMIENTO DEL AGENTE

Si la casilla está limpia, el agente se mueve a la otra casilla para evaluar si requiere limpieza, asegurando un proceso efectivo y continuo de mantenimiento.

Resultados de la Simulación del Agente

SIMULACIONES EJECUTADAS

Se realizaron tres simulaciones distintas, cada una con diferentes niveles de suciedad en las casillas A y B, evaluando la efectividad del agente en cada caso.

ÉXITO DEL AGENTE

En todos los escenarios, el agente logró limpiar ambas casillas, demostrando su eficiencia y la efectividad de las reglas implementadas en el algoritmo de limpieza.

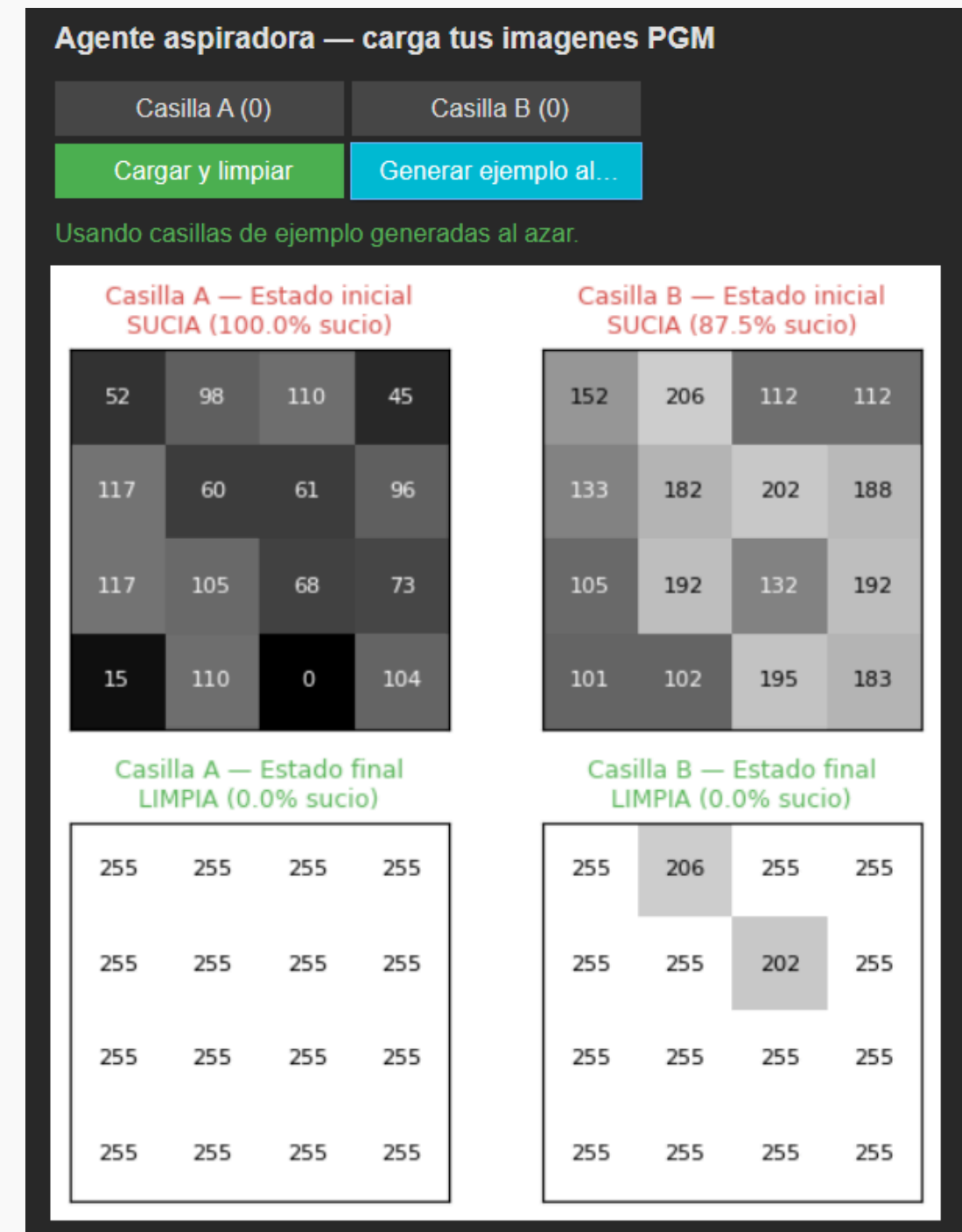
DATOS REGISTRADOS

Durante las simulaciones, se registraron las iteraciones, los movimientos realizados por el agente y la cantidad de píxeles limpiados, proporcionando un análisis detallado del proceso.

Ejemplo de funcionamiento

SIMULACION PASO A PASO

- Cada casilla (A y B) se muestra como una cuadrícula de 4x4 píxeles en escala de grises, con su valor numérico (0-255) sobre cada celda.
- Estado inicial: se ve el tono de gris original y una etiqueta de color — rojo si está SUCIA (>50% de píxeles por debajo del umbral 200), verde si está LIMPIA.
- El agente aplica su regla: si la casilla está sucia, la limpia (sube todos los píxeles sucios a 255); si ya está limpia, se mueve a la otra casilla.
- Estado final: ambas casillas terminan en blanco puro (255), es decir, 0% de suciedad — el ambiente queda completamente limpio.



Ejemplo de funcionamiento

RESPUESTA DE EJECUCION

Metricas de desempeno del agente

Iteraciones totales	3
Movimientos realizados	1
Acciones de limpieza	2
Pixeles limpiados en total	30
Suciedad final Casilla A	0.0%
Suciedad final Casilla B	0.0%
Superficie limpia total	100.0%

- Iteraciones totales (3): cuántas veces el agente "pensó y actuó" antes de terminar (percibir → decidir → actuar, una vez por iteración).
- Movimientos realizados (1): cuántas veces el agente se desplazó de una casilla a la otra (solo se mueve cuando la casilla donde está ya quedó limpia).
- Acciones de limpieza (2): cuántas veces ejecutó la acción de limpiar (una por cada casilla que encontró sucia: A y B).
- Pixeles limpiados en total (30): la suma de todos los pixeles individuales que cambiaron de "sucio" a 255 (blanco) en ambas casillas.
- Suciedad final Casilla A / B (0.0%): confirma que, al terminar, ninguna de las dos casillas tiene pixeles sucios.
- Superficie limpia total (100.0%): el promedio de limpieza entre A y B — indica que el ambiente completo quedó limpio.

Ejemplo de funcionamiento

```
def ejecutar_simulacion(casilla_A, casilla_B):
    """
    Ejecuta el agente reactivo simple sobre las casillas A y B.
    Devuelve un diccionario con todo lo necesario para pintar la interfaz
    grafica: estados inicial/final de cada casilla y las metricas de desempeño.
    """
    estado_inicial = {"A": copiar_casilla(casilla_A), "B": copiar_casilla(casilla_B)}

    ubicacion = "A"
    casillas = {"A": casilla_A, "B": casilla_B}

    movimientos = 0
    acciones_limpieza = 0
    pixeles_limpiados_total = 0
    iteracion = 0
    log = []

    while casilla_esta_sucia(casillas["A"]) or casilla_esta_sucia(casillas["B"]):
        iteracion += 1
        if iteracion > MAX_ITERACIONES:
            log.append("Se alcanzo el límite de iteraciones de seguridad.")
            break

        casilla_actual = casillas[ubicacion]
        suciedad_actual = calcular_porcentaje_sucio(casilla_actual)

        if casilla_esta_sucia(casilla_actual):
            _, limpiados = limpiar_casilla(casilla_actual)
            acciones_limpieza += 1
            pixeles_limpiados_total += limpiados
            log.append(
                f"Iteracion {iteracion}: Casilla {ubicacion} sucia "
                f"({suciedad_actual:.1f}%) -> LIMPIAR ({limpiados} pixeles)."
            )
        else:
            nueva_ubicacion = "B" if ubicacion == "A" else "A"
            log.append(f"Iteracion {iteracion}: Casilla {ubicacion} limpia -> MOVERSE a Casilla {nueva_ubicacion}.")
            ubicacion = nueva_ubicacion
            movimientos += 1

    porcentaje_final_A = calcular_porcentaje_sucio(casillas["A"])
    porcentaje_final_B = calcular_porcentaje_sucio(casillas["B"])
    superficie_limpia_total = 100 - ((porcentaje_final_A + porcentaje_final_B) / 2)
```

RESPUESTA DE EJECUCION

- Guarda una copia de cómo llegaron A y B (para poder mostrar "antes" al final).
- El agente arranca parado en la Casilla A.
- Entra en un ciclo (while) que se repite mientras A o B sigan sucias:
- Mira la casilla donde está parado.
- Si está sucia → la limpia (todos sus pixeles suben a 255) y suma 1 a "acciones de limpieza".
- Si ya está limpia → se mueve a la otra casilla y suma 1 a "movimientos".
- Cada paso queda anotado en un log de texto.
- El ciclo termina cuando ambas casillas quedan limpias (o si se alcanza el límite de seguridad de 30 iteraciones).
- Calcula las métricas finales (iteraciones, movimientos, limpiezas, % de suciedad final) y devuelve todo junto para que la interfaz lo dibuje.